20 projets logiciels diversifiés : stack, fonctionnalités, compétences et phases

Catalogue de 20 projets web et mobile complexes, chacun avec sa stack justifiée, ses fonctionnalités, les compétences acquises et un plan de développement par phases documentées.

Comment lire ce catalogue

Ce document propose 20 projets logiciels complexes et volontairement diversifiés (web, mobile, temps réel, IoT, IA, données, jeu). Chaque projet est décrit individuellement : objectif et exigences, stack technique justifiée (pourquoi ce langage, pourquoi ce framework), fonctionnalités à implémenter, compétences acquises et mises en valeur sur un CV, puis phases de développement avec les documents à produire à chaque phase. Les documents de phase s'assemblent à la fin pour former la documentation générale du projet et servir de base au guide utilisateur.

Projets

20

12 web · 8 mobile ou hybrides

Langages couverts

12

TypeScript, Python, Java, Kotlin, Swift, Dart, Go, Rust, C#, PHP, Ruby, Elixir

Ordre conseillé. Les projets sont classés par difficulté croissante approximative à l'intérieur de trois niveaux : *Intermédiaire* (1 à 6), *Avancé* (7 à 14), *Expert* (15 à 20). Commencez par un projet dont vous maîtrisez déjà au moins un élément de la stack, puis alternez les langages pour élargir votre profil.

Méthodologie commune : 7 phases, 7 documents

Tous les projets suivent le même cycle. Chaque phase se termine par un document versionné dans le dépôt (dossier `docs/`). La concaténation de ces documents constitue la documentation générale ; les documents des phases 2, 6 et 7 alimentent directement le guide utilisateur.

Phases de développement et document livré à chaque phase

Phase	Activités	Document produit	Contribue au guide utilisateur ?
0. Cadrage	Problème, cibles, périmètre, contraintes, indicateurs de succès, risques.	`docs/00-cahier-des-charges.md`	Non (contexte)
1. Analyse	Personas, user stories, règles métier, diagramme de cas d'utilisation, MoSCoW.	`docs/01-specifications-fonctionnelles.md`	Oui : liste des fonctionnalités et parcours
2. Conception	Architecture (C4), modèle de données, contrat d'API (OpenAPI / GraphQL / proto), maquettes (Figma), choix techniques argumentés (ADR).	`docs/02-architecture.md`, `docs/adr/`, `api/openapi.yaml`	Oui : maquettes et vocabulaire
3. Mise en place	Dépôt mono ou multi, CI (lint, tests, build), Docker Compose, gestion des secrets, conventions de commit, environnements.	`README.md`, `docs/03-guide-developpeur.md`	Non
4. Développement itératif	Sprints de 1 à 2 semaines ; chaque fonctionnalité = branche, tests, revue, note de version.	`CHANGELOG.md`, `docs/04-journal-de-sprints.md`	Oui : notes de version
5. Qualité et sécurité	Tests unitaires, intégration, E2E, charge, audit OWASP, accessibilité, performance.	`docs/05-strategie-de-tests.md`, rapport de tests	Non
6. Déploiement et exploitation	Infra as code, pipeline de release, monitoring, sauvegardes, plan de reprise, publication stores.	`docs/06-guide-deploiement.md`, `docs/06-runbook.md`	Oui : installation, FAQ, dépannage
7. Documentation finale	Assemblage, captures d'écran, parcours pas à pas, glossaire, FAQ, vidéo de démonstration.	`docs/07-documentation-generale.md`, `docs/guide-utilisateur.md`	Oui : c'est le guide lui-même

+ Structure type du guide utilisateur (à réutiliser pour chaque projet)

Vue d'ensemble : quel langage pour quel type de projet

Les 20 projets, leur type, leur stack principale et le niveau visé

N°	Projet	Type	Langages et frameworks	Niveau
01	Gestion scolaire (notes, absences, bulletins)	Web	PHP · Laravel · Vue.js · MySQL	Intermédiaire
02	Marketplace immobilière géolocalisée	Web	Ruby · Rails · Hotwire · PostGIS · Elasticsearch	Intermédiaire
03	SaaS de facturation et gestion PME multi-tenant	Web	Python · Django · DRF · React · Celery	Intermédiaire
04	Application agricole hors ligne (agritech)	Mobile Android	Kotlin · Jetpack Compose · Room · FastAPI	Intermédiaire
05	Finances personnelles avec catégorisation IA	Mobile iOS	Swift · SwiftUI · Core Data · FastAPI · scikit-learn	Intermédiaire
06	Billetterie événementielle avec QR anti-fraude	Web + mobile	Go · gRPC · React · Flutter	Intermédiaire
07	E-commerce multi-vendeurs	Web	TypeScript · Next.js · NestJS · PostgreSQL · Stripe	Avancé
08	Portefeuille et paiement mobile (fintech)	Mobile + backend	Kotlin · Java · Spring Boot · PostgreSQL	Avancé
09	Télémédecine (rendez-vous, visio, dossier patient)	Mobile multiplateforme	Dart · Flutter · Django · WebRTC	Avancé
10	Livraison de repas (client, livreur, restaurant)	Mobile + web	Flutter · Go · PostGIS · Redis	Avancé
11	Kanban collaboratif temps réel et hors ligne	Web	C# · ASP.NET Core · SignalR · Angular	Avancé
12	Portail e-gouvernement (démarches, files, paiements)	Web	Java · Spring Boot · Angular · Keycloak	Avancé
13	Tableau de bord BI avec pipelines de données	Web + data	Python · Airflow · FastAPI · ClickHouse · React	Avancé
14	Streaming vidéo (upload, transcodage, HLS)	Web	TypeScript · Next.js · Node.js · FFmpeg · BullMQ	Avancé
15	Réseau social et messagerie temps réel	Mobile + backend	Elixir · Phoenix · React Native	Expert
16	Domotique IoT temps réel	IoT + web + mobile	Rust · Actix · MQTT · TimescaleDB · Svelte · Kotlin	Expert
17	Covoiturage / VTC avec suivi GPS	Mobile multiplateforme	Kotlin Multiplatform · Node.js · Socket.IO · Redis Geo	Expert

N°	Projet	Type	Langages et frameworks	Niveau
18	Réservation voyage en microservices	Web distribué	Java · Spring Cloud · Kafka · Kubernetes · React	Expert
19	Jeu mobile multijoueur temps réel	Jeu mobile	C# · Unity · Go · WebSocket · Redis	Expert
20	Assistant IA de service client (RAG)	Web + IA	Python · FastAPI · LangChain · pgvector · React	Expert

+ Règles générales d'adéquation langage / type de projet

01 · Système de gestion scolaire

Type	Application web multi-rôles
Domaine	Éducation
Niveau	Intermédiaire
Durée indicative	6 à 8 semaines

Objectif et exigences

Digitaliser la vie d'un établissement : inscriptions, emplois du temps, saisie des notes, absences, calcul de moyennes et génération de bulletins PDF, communication parents-école. Exigences : quatre rôles (administration, enseignant, élève, parent), calcul de moyennes paramétrable par coefficient, export PDF fidèle, fonctionnement sur connexion faible, RGPD (données de mineurs).

Stack et justification

Choix techniques du projet 01

Couche	Technologie	Pourquoi
Backend	PHP 8 · Laravel 11	Framework complet (Eloquent ORM, auth, files d'attente, politiques d'autorisation, génération PDF via DomPDF) qui permet de livrer un back-office CRUD riche très rapidement ; hébergement mutualisé bon marché, adapté aux écoles.
Frontend	Vue.js 3 + Inertia.js	Inertia relie Laravel et Vue sans écrire d'API REST séparée : on garde la simplicité d'une app monolithique avec l'ergonomie d'une SPA (saisie de notes en grille, filtres instantanés).
Base de données	MySQL 8	Modèle relationnel classique (classes, matières, élèves, notes), transactions simples, très répandu chez les hébergeurs PHP.
Outils	Laravel Excel, Laravel Scheduler, Mailgun	Import massif d'élèves, tâches planifiées (rappels d'absences), e-mails transactionnels.

Fonctionnalités

- Gestion des années scolaires, classes, matières, coefficients et enseignants.
- Inscription des élèves avec import Excel et génération de matricules.
- Emploi du temps hebdomadaire avec détection de conflits de salle / enseignant.
- Saisie des notes en grille, calcul des moyennes, rangs et appréciations.
- Gestion des absences et retards avec notification SMS / e-mail aux parents.
- Génération de bulletins PDF par lot et archivage.
- Espace parent : consultation des notes, absences, messagerie interne.
- Tableau de bord administratif (taux de réussite, effectifs, paiements de scolarité).
- Journal d'audit des modifications de notes.

Compétences acquises

- Modélisation relationnelle et migrations Laravel ; requêtes Eloquent optimisées (N+1).
- Autorisation fine (Policies, Gates) et gestion multi-rôles.
- Génération de documents PDF et exports Excel.
- Architecture monolithe moderne (Inertia) et composants Vue réutilisables.
- Tests PHPUnit / Pest, factories et seeders ; conformité RGPD.

Phases de développement

Phases spécifiques du projet 01

Phase	Étapes clés	Document
0-1	Interviews d'un enseignant et d'un administrateur ; user stories par rôle ; règles de calcul des moyennes.	Cahier des charges, spécifications fonctionnelles
2	MCD/MLD (20 tables), maquettes des grilles de saisie, choix Inertia (ADR-001).	Architecture, ADR, dictionnaire de données
3	Laravel Sail (Docker), GitHub Actions (Pint, Pest), seeders réalistes.	Guide développeur
4	Sprint 1 : structure école et rôles · Sprint 2 : notes et moyennes · Sprint 3 : absences et notifications · Sprint 4 : bulletins PDF et espace parent.	Journal de sprints, changelog
5	Tests de calcul de moyennes (cas limites), tests de politiques, tests E2E Dusk.	Stratégie de tests
6	Déploiement Forge ou VPS + Nginx, sauvegardes MySQL quotidiennes.	Guide de déploiement, runbook
7	Guide utilisateur en 4 chapitres (un par rôle), vidéo de saisie de notes.	Documentation générale, guide utilisateur

02 · Marketplace immobilière géolocalisée

Type	Application web
Domaine	Immobilier
Niveau	Intermédiaire
Durée indicative	6 à 8 semaines

Objectif et exigences

Plateforme de petites annonces immobilières (vente, location) avec recherche sur carte, filtres avancés, messagerie acheteur-agent et gestion des visites. Exigences : recherche par polygone dessiné sur la carte, résultats en moins de 300 ms, upload de photos optimisées, SEO (pages d'annonces indexables), modération des annonces.

Stack et justification

Choix techniques du projet 02

Couche	Technologie	Pourquoi
Backend	Ruby 3 · Rails 7	Convention plutôt que configuration : Active Record, Active Storage (photos), Action Mailer, Devise pour l'auth. Rails reste l'un des moyens les plus rapides de construire une marketplace complète et rendue côté serveur (bon pour le SEO).
Frontend	Hotwire (Turbo + Stimulus) · Leaflet	Interactivité (filtres, pagination, favoris) sans SPA lourde ; Leaflet pour la carte et le dessin de zones.
Données	PostgreSQL + PostGIS	Requêtes géospatiales natives (ST_Within, ST_DWithin) pour la recherche par zone et par rayon.
Recherche	Elasticsearch (ou Meilisearch)	Recherche plein texte, facettes (prix, surface, pièces) et tri par pertinence, plus rapide que SQL pour les filtres combinés.
Traitement	Sidekiq + Redis, libvips	Redimensionnement d'images et indexation en arrière-plan.

Fonctionnalités

- Comptes particuliers et agences, abonnements et annonces mises en avant.
- Dépôt d'annonce en plusieurs étapes avec géocodage automatique de l'adresse.
- Recherche par carte (zone dessinée, rayon), filtres à facettes, tri, sauvegarde de recherche avec alertes e-mail.
- Galerie photos optimisée, visite virtuelle (lien 360°), estimation de prix au m² par quartier.
- Messagerie interne et prise de rendez-vous de visite avec calendrier.
- Favoris, comparateur d'annonces, historique de prix.
- Back-office de modération, signalements, statistiques d'audience.
- Pages SEO (sitemap, données structurées schema.org, URLs lisibles).

Compétences acquises

- Requêtes géospatiales PostGIS et intégration cartographique.
- Moteur de recherche à facettes et synchronisation d'index.
- Rails moderne (Hotwire, Active Storage, Action Cable pour la messagerie).
- Traitement asynchrone (Sidekiq) et optimisation d'images.
- SEO technique, RSpec, Capybara, performance (cache fragments, index).

Phases de développement

Phases spécifiques du projet 02

Phase	Étapes clés	Document
0-1	Benchmark de 3 sites concurrents, personas acheteur / agent / modérateur, priorisation MoSCoW.	Cahier des charges, spécifications
2	Modèle de données avec colonnes géographiques, schéma d'index Elasticsearch, maquettes carte + liste.	Architecture, ADR (Hotwire vs React), dictionnaire de données
3	Docker Compose (Postgres/PostGIS, ES, Redis), RuboCop, RSpec, CI.	Guide développeur
4	Sprint 1 : auth et annonces · Sprint 2 : carte et recherche · Sprint 3 : messagerie et visites · Sprint 4 : modération, SEO, alertes.	Journal de sprints
5	Tests de requêtes géo, tests de charge de la recherche (k6), audit Lighthouse.	Stratégie et rapport de tests
6	Déploiement Kamal ou Render, CDN images, sauvegardes.	Guide de déploiement, runbook
7	Guide « déposer une annonce », « rechercher sur la carte », « modérer ».	Documentation générale, guide utilisateur

03 · SaaS de facturation et gestion PME multi-tenant

Type	Application web SaaS
Domaine	Gestion d'entreprise / comptabilité
Niveau	Intermédiaire à avancé
Durée indicative	8 semaines

Objectif et exigences

Permettre à des PME de gérer clients, devis, factures, stocks et relances dans un espace isolé par entreprise (tenant). Exigences : isolation stricte des données, numérotation légale des factures (séquentielle, sans trou), TVA multi-taux, PDF conformes, export comptable, plans d'abonnement et facturation du SaaS lui-même.

Stack et justification

Choix techniques du projet 03

Couche	Technologie	Pourquoi
Backend	Python 3.12 · Django 5 · Django REST Framework	ORM mature avec transactions, admin intégré pour le support, système d'auth et de permissions robuste ; DRF pour une API propre consommée par le frontend et par des intégrations tierces.
Multi-tenant	django-tenants ou colonne `tenant_id` + Row Level Security PostgreSQL	Deux stratégies à comparer et documenter ; la RLS garantit l'isolation même en cas de bug applicatif.
Frontend	React 18 · TypeScript · TanStack Query · React Hook Form	Formulaires complexes (lignes de facture dynamiques), cache de données, typage des DTO générés depuis OpenAPI.
Asynchrone	Celery + Redis	Génération de PDF, envoi de relances, exports lourds, tâches planifiées (Celery Beat).
Paiement	Stripe Billing	Abonnements du SaaS, webhooks, portail client.

Fonctionnalités

- Onboarding d'entreprise, invitation d'utilisateurs, rôles (gérant, comptable, commercial).
- Catalogue produits / services, gestion de stock avec mouvements et alertes de seuil.
- Devis convertibles en factures, avoirs, acomptes, factures récurrentes.
- Numérotation séquentielle verrouillée, TVA multi-taux, devises, mentions légales.
- PDF personnalisables (logo, couleurs), envoi par e-mail avec suivi d'ouverture.
- Relances automatiques, échéancier, rapprochement des paiements.
- Tableaux de bord (CA, impayés, top clients), export comptable CSV / FEC.
- API publique avec clés par tenant et webhooks sortants.
- Abonnements Stripe, quotas par plan, page de facturation du SaaS.

Compétences acquises

- Architecture multi-tenant et sécurité au niveau base de données (RLS).
- Intégrité transactionnelle (verrous, `select_for_update`) pour la numérotation.
- Conception d'API REST versionnée, documentation OpenAPI, SDK généré.
- Traitement asynchrone et planifié avec Celery.
- Intégration Stripe (webhooks idempotents), tests pytest, factories, couverture.

Phases de développement

Phases spécifiques du projet 03

Phase	Étapes clés	Document
0-1	Règles de facturation légales du pays cible, personas, parcours devis → facture → paiement.	Cahier des charges, spécifications, règles métier
2	ADR multi-tenant (schéma par tenant vs RLS), modèle de données comptable, contrat OpenAPI, maquettes.	Architecture, ADR, OpenAPI
3	Docker Compose, pre-commit (ruff, mypy), pytest, CI, génération du client TypeScript.	Guide développeur
4	Sprint 1 : tenants et auth · Sprint 2 : clients, produits, devis · Sprint 3 : factures, PDF, numérotation · Sprint 4 : relances, dashboards · Sprint 5 : Stripe et API publique.	Journal de sprints
5	Tests d'isolation inter-tenant, tests de concurrence sur la numérotation, audit OWASP.	Stratégie de tests, rapport de sécurité
6	Déploiement sur VPS ou Fly.io, Sentry, sauvegardes chiffrées, procédure de restauration testée.	Guide de déploiement, runbook
7	Guide par rôle, FAQ facturation, documentation API pour intégrateurs.	Documentation générale, guide utilisateur, portail API

04 · Application agricole hors ligne (agritech)

Type	Application mobile Android native + API
Domaine	Agriculture, zones à faible connectivité
Niveau	Intermédiaire
Durée indicative	6 à 8 semaines

Objectif et exigences

Outil de suivi d'exploitation pour petits producteurs : parcelles, cultures, intrants, récoltes, prix du marché et conseils, utilisable sans réseau puis synchronisé. Exigences : fonctionnement 100 % hors ligne, synchronisation fiable avec résolution de conflits, faible consommation de données, interface multilingue avec icônes et audio pour publics peu alphabétisés, appareils d'entrée de gamme.

Stack et justification

Choix techniques du projet 04

Couche	Technologie	Pourquoi
Mobile	Kotlin · Jetpack Compose · Room · WorkManager · DataStore	Android domine le marché cible ; Room offre une base SQLite typée pour le hors ligne, WorkManager garantit la synchronisation différée même après redémarrage ; Compose accélère les UI adaptatives.
Architecture	MVVM + Clean Architecture · Hilt · Coroutines/Flow	Séparation UI / domaine / données, injection de dépendances, flux réactifs de la base vers l'écran.
Backend	Python · FastAPI · SQLAlchemy · PostgreSQL	API asynchrone légère, validation Pydantic, documentation automatique ; Python facilite l'ajout ultérieur de modèles de prédiction de rendement.
Synchronisation	Protocole delta (horodatage / version par ligne), UUID côté client	Évite les collisions d'identifiants et permet une fusion par « dernière écriture gagne » ou par champ.
Services	API météo publique, SMS gateway	Alertes météo et prix par SMS pour les utilisateurs sans smartphone connecté.

Fonctionnalités

- Cartographie de parcelles par marche GPS (polygone), calcul de surface.
- Calendrier cultural par culture, rappels de tâches (semis, traitement, récolte).
- Journal des intrants et dépenses, récoltes et ventes ; marge par parcelle.
- Prix du marché par produit et localité, saisie collaborative, graphiques.
- Fiches conseils (texte, image, audio) téléchargées en cache.
- Synchronisation en arrière-plan avec file d'attente et indicateur d'état.
- Multilingue (français + langues locales), mode sombre, gros boutons.
- Back-office web minimal (FastAPI + templates) pour publier conseils et prix.

Compétences acquises

- Conception offline-first, synchronisation et gestion des conflits.
- Android moderne : Compose, Room, WorkManager, Hilt, Flow.
- Géolocalisation et calculs géométriques sur mobile.
- API FastAPI typée, tests Pytest, migrations Alembic.
- Accessibilité et design inclusif, optimisation pour appareils modestes.

Phases de développement

Phases spécifiques du projet 04

Phase	Étapes clés	Document
0-1	Enquête terrain (5 producteurs), contraintes réseau, personas, parcours hors ligne.	Cahier des charges, spécifications
2	Schéma Room et PostgreSQL alignés, protocole de sync (ADR), maquettes basse fidélité testées sur papier.	Architecture, ADR sync, OpenAPI
3	Projet Android modulaire, ktlint/detekt, tests instrumentés, CI GitHub Actions ; backend Docker.	Guide développeur
4	Sprint 1 : parcelles et cultures locales · Sprint 2 : journal et rappels · Sprint 3 : sync et backend · Sprint 4 : prix, conseils, SMS.	Journal de sprints
5	Tests de sync avec coupures réseau simulées, tests sur appareils bas de gamme, profilage batterie.	Stratégie de tests
6	Publication Play Store (interne → ouverte), backend sur VPS, monitoring.	Guide de déploiement, fiche Play Store
7	Guide illustré et audio, affiche « premiers pas » imprimable.	Documentation générale, guide utilisateur

05 · Finances personnelles avec catégorisation intelligente

Type	Application mobile iOS native + API
Domaine	Finance personnelle / IA
Niveau	Intermédiaire à avancé
Durée indicative	7 semaines

Objectif et exigences

Aider l'utilisateur à suivre ses dépenses, fixer des budgets et atteindre des objectifs d'épargne, avec catégorisation automatique des transactions par apprentissage automatique. Exigences : import de relevés CSV / OFX et saisie manuelle, données chiffrées, fonctionnement hors ligne, widgets et notifications, respect des guidelines Apple.

Stack et justification

Choix techniques du projet 05

Couche	Technologie	Pourquoi
Mobile	Swift 5.10 · SwiftUI · Swift Charts · Core Data / SwiftData · WidgetKit	Expérience native (Face ID, widgets, Siri Shortcuts), graphiques natifs performants, persistance locale chiffrée ; SwiftUI réduit fortement le code d'interface.
Architecture	MVVM + The Composable Architecture (optionnel) · async/await	État prévisible et testable, concurrence structurée.
Backend / ML	Python · FastAPI · scikit-learn · pandas	Entraînement d'un classifieur de libellés (TF-IDF + régression logistique) et service de prédiction ; export du modèle en Core ML pour l'inférence sur l'appareil.
Données	PostgreSQL, CloudKit (option sync)	Sauvegarde et synchronisation multi-appareils.

Fonctionnalités

- Comptes multiples, saisie rapide de transactions, transactions récurrentes.
- Import CSV / OFX avec détection de doublons.
- Catégorisation automatique avec correction manuelle qui réentraîne le modèle.
- Budgets mensuels par catégorie, alertes de dépassement, objectifs d'épargne.
- Graphiques (tendances, répartition, comparaison de mois) et rapports PDF.
- Widgets écran d'accueil, verrouillage biométrique, export de données.
- Mode multi-devises et taux de change mis en cache.

Compétences acquises

- Développement iOS natif complet et publication App Store.
- Persistance locale, chiffrement, sécurité mobile (Keychain).
- Cycle de vie d'un modèle ML : données, entraînement, évaluation, déploiement sur mobile (Core ML).
- API FastAPI et pipeline de réentraînement.

- Tests XCTest / UI tests, TestFlight, accessibilité VoiceOver.

Phases de développement

Phases spécifiques du projet 05

Phase	Étapes clés	Document
0-1	Personas (étudiant, salarié), user stories, liste de catégories, contraintes de confidentialité.	Cahier des charges, spécifications
2	Modèle de données, schéma du pipeline ML, ADR « inférence locale vs serveur », maquettes SwiftUI.	Architecture, ADR, fiche modèle ML
3	Projet Xcode modulaire (SPM), SwiftLint, fastlane, CI ; environnement Python (uv/poetry).	Guide développeur
4	Sprint 1 : comptes et transactions · Sprint 2 : import et budgets · Sprint 3 : ML et Core ML · Sprint 4 : graphiques, widgets, sécurité.	Journal de sprints
5	Tests unitaires du parseur OFX, évaluation du modèle (précision, matrice de confusion), tests UI.	Stratégie de tests, rapport d'évaluation ML
6	TestFlight, App Store Connect, politique de confidentialité, backend déployé.	Guide de déploiement
7	Guide utilisateur avec captures, FAQ « pourquoi cette catégorie ? ».	Documentation générale, guide utilisateur

06 · Billetterie événementielle avec QR codes anti-fraude

Type	Web (vente, back-office) + mobile (contrôle d'accès)
Domaine	Événementiel
Niveau	Intermédiaire à avancé
Durée indicative	7 semaines

Objectif et exigences

Vendre des billets en ligne pour des concerts et conférences et contrôler les entrées par scan. Exigences : pas de survente lors de pics (10 000 acheteurs simultanés), billets infalsifiables (signature cryptographique), scan hors ligne au portique, remboursement et transfert de billet, statistiques en direct.

Stack et justification

Choix techniques du projet 06

Couche	Technologie	Pourquoi
Backend	Go 1.22 · gRPC + gRPC-Gateway (REST) · sqlc	Concurrence native (goroutines) et faible latence pour absorber les pics ; binaire unique facile à déployer ; gRPC pour l'app de scan, REST généré pour le web.
Inventaire	Redis (scripts Lua atomiques) + PostgreSQL	Décrément atomique du stock en mémoire avec réservation temporaire, persistance et réconciliation en base.
Web	React · Vite · TypeScript	Front de vente et back-office organisateur.
App de scan	Flutter (Android + iOS)	Une base de code pour les deux plateformes, accès caméra, base locale SQLite pour le scan hors ligne.
Sécurité	Billets signés (Ed25519 / JWT), clé publique embarquée dans l'app	Vérification hors ligne sans appel serveur ; détection de doublons synchronisée ensuite.
Paiement	Stripe / Mobile Money API	Paiement carte et mobile money selon le marché.

Fonctionnalités

- Création d'événements, catégories de billets, quotas, codes promo, plan de salle simplifié.
- File d'attente virtuelle et réservation temporaire de 10 minutes.
- Paiement, e-billet PDF et wallet (Apple / Google Wallet), envoi par e-mail / SMS.
- Transfert nominatif et remboursement partiel selon la politique de l'événement.
- Application de scan : validation hors ligne, comptage des entrées, alertes de doublon, multi-portiques.
- Tableau de bord organisateur en temps réel (ventes, entrées, revenus).
- Exports, reversements aux organisateurs, journal d'audit.

Compétences acquises

- Programmation concurrente en Go, gestion de la contention et idempotence.
- Conception de protocoles gRPC / Protobuf et génération de code.
- Cryptographie appliquée (signatures, rotation de clés).
- Flutter : caméra, stockage local, synchronisation.
- Tests de charge (k6, Vegeta), observabilité (Prometheus, Grafana).

Phases de développement

Phases spécifiques du projet 06

Phase	Étapes clés	Document
0-1	Scénarios de fraude, objectifs de charge, personas acheteur / organisateur / contrôleur.	Cahier des charges, spécifications, modèle de menaces
2	Fichiers .proto, algorithme de réservation (ADR Redis Lua), format du billet signé, maquettes.	Architecture, ADR, contrat gRPC
3	Monorepo (Go, React, Flutter), buf, golangci-lint, CI multi-plateformes.	Guide développeur
4	Sprint 1 : événements et catalogue · Sprint 2 : inventaire et paiement · Sprint 3 : billets signés et app de scan · Sprint 4 : dashboards, transferts, remboursements.	Journal de sprints
5	Test de charge « 10 000 acheteurs / 100 billets », tests de scan hors ligne, revue de sécurité.	Rapport de charge, stratégie de tests
6	Déploiement conteneurisé (Docker + Traefik ou Kubernetes), publication de l'app de scan.	Guide de déploiement, runbook « jour J »
7	Guides organisateur et contrôleur, FAQ acheteur.	Documentation générale, guide utilisateur

07 · Plateforme e-commerce multi-vendeurs

Type	Application web (boutique, espace vendeur, admin)
Domaine	Commerce en ligne
Niveau	Avancé
Durée indicative	10 semaines

Objectif et exigences

Marketplace où plusieurs vendeurs publient des produits, gèrent stocks et commandes, et où la plateforme encaisse puis reverse les paiements. Exigences : catalogue avec variantes (taille, couleur), recherche rapide, panier persistant, paiement sécurisé avec répartition (Stripe Connect), SEO et performance (Core Web Vitals), notifications de commande, avis vérifiés.

Stack et justification

Choix techniques du projet 07

Couche	Technologie	Pourquoi
Frontend	TypeScript · Next.js 14 (App Router) · Tailwind CSS · shadcn/ui	Rendu serveur et statique pour le SEO des pages produit, streaming, images optimisées ; TypeScript partagé avec le backend (types des DTO).
Backend	NestJS · Prisma · PostgreSQL	Architecture modulaire (modules, providers, guards) inspirée d'Angular, injection de dépendances, validation par décorateurs ; Prisma pour un accès typé aux données.
Recherche et cache	Meilisearch · Redis	Recherche tolérante aux fautes et facettes ; cache de sessions, paniers et rate limiting.
Paiement	Stripe Connect	Encaissement, commission de la plateforme et reversement automatique aux vendeurs.
Messagerie	BullMQ, Resend / SendGrid	E-mails transactionnels, traitement asynchrone des commandes, webhooks.
Stockage	S3 compatible (MinIO en local)	Images produits avec URL pré-signées.

Fonctionnalités

- Inscription vendeur avec validation KYC Stripe, boutique personnalisée.
- Produits avec variantes, attributs, stock par variante, import CSV.
- Recherche instantanée, filtres à facettes, pages catégories et collections.
- Panier persistant (invité et connecté), codes promo, frais de port par zone.
- Checkout Stripe avec 3D Secure, répartition multi-vendeurs, factures.
- Suivi de commande, statuts, étiquettes d'expédition, retours.
- Avis vérifiés, questions / réponses, wishlist.
- Espace vendeur : commandes, statistiques, reversements ; admin : modération, commissions, litiges.

- Internationalisation (langue, devise) et pages SEO (sitemap, données structurées).

Compétences acquises

- Full-stack TypeScript de bout en bout, monorepo (Turborepo) et partage de types.
- Rendu serveur, ISR, optimisation des performances web.
- Architecture modulaire NestJS, Prisma, transactions et migrations.
- Paiements complexes (Connect, webhooks idempotents, litiges).
- Tests Jest, Playwright E2E, observabilité (OpenTelemetry).

Phases de développement

Phases spécifiques du projet 07

Phase	Étapes clés	Document
0-1	Étude de modèles de commission, personas acheteur / vendeur / admin, parcours d'achat.	Cahier des charges, spécifications
2	Modèle produit / variante / stock, machine à états des commandes, ADR Next.js + NestJS, contrat OpenAPI, maquettes.	Architecture, ADR, OpenAPI
3	Turborepo, ESLint / Prettier, Docker Compose (Postgres, Redis, Meilisearch, MinIO), CI.	Guide développeur
4	Sprint 1 : auth et catalogue · Sprint 2 : recherche et panier · Sprint 3 : checkout Stripe Connect · Sprint 4 : commandes et expéditions · Sprint 5 : espace vendeur et admin · Sprint 6 : avis, i18n, SEO.	Journal de sprints
5	E2E Playwright du parcours d'achat, tests de webhooks, Lighthouse CI, audit OWASP.	Stratégie de tests
6	Vercel (front) + Fly.io / Railway (API) ou Kubernetes, CDN, sauvegardes.	Guide de déploiement, runbook
7	Guides acheteur, vendeur, administrateur ; centre d'aide.	Documentation générale, guide utilisateur

08 · Portefeuille et paiement mobile (fintech)

Type	Application mobile Android + backend bancaire
Domaine	Finance / paiements
Niveau	Avancé
Durée indicative	10 semaines

Objectif et exigences

Portefeuille électronique : dépôt, retrait, transfert entre utilisateurs, paiement marchand par QR, historique et plafonds. Exigences : comptabilité en partie double (aucun argent créé ou perdu), idempotence des transactions, KYC par niveaux, authentification forte (PIN + biométrie + OTP), chiffrement, traçabilité complète, disponibilité élevée.

Stack et justification

Choix techniques du projet 08

Couche	Technologie	Pourquoi
Backend	Java 21 · Spring Boot 3 · Spring Security · Spring Data JPA	Référence du secteur bancaire : transactions ACID, gestion fine des isolations, écosystème de sécurité (OAuth2, JWT), maturité, forte employabilité.
Base de données	PostgreSQL	Contraintes fortes, verrous, `SERIALIZABLE` pour le grand livre ; tables d'écritures immuables.
Mobile	Kotlin · Jetpack Compose · Biometric API · CameraX (QR)	Accès natif à la biométrie, au stockage sécurisé (Keystore) et à la caméra ; détection de root et attestation Play Integrity.
Messagerie	RabbitMQ ou Kafka	Notifications, journal d'audit et intégration avec agrégateurs externes de façon asynchrone.
Sécurité	Keycloak, Vault, TLS pinning	Gestion des identités, des secrets et protection contre l'interception.

Fonctionnalités

- Inscription par numéro de téléphone, OTP, KYC (pièce d'identité, selfie) par niveaux avec plafonds associés.
- Dépôt / retrait via agents ou agrégateur mobile money simulé.
- Transfert P2P instantané, demande de paiement, paiement marchand par QR dynamique.
- Grand livre en partie double, soldes calculés, réconciliation quotidienne.
- Historique filtrable, reçus PDF, export.
- Sécurité : PIN, biométrie, détection d'appareil inconnu, blocage, limites anti-fraude (règles + score).
- Back-office : supervision des transactions, gestion des agents, gel de compte, reporting réglementaire.
- Notifications push (FCM) et SMS.

Compétences acquises

- Modélisation comptable (ledger) et garanties transactionnelles.
- Spring Security avancé, OAuth2, gestion des secrets, durcissement d'API.
- Sécurité mobile (Keystore, attestation, obfuscation).
- Architecture événementielle et idempotence.
- Tests d'intégration Testcontainers, JUnit 5, tests de propriété sur le ledger.

Phases de développement

Phases spécifiques du projet 08

Phase	Étapes clés	Document
0-1	Cadre réglementaire (KYC/AML), plafonds, personas client / agent / marchand / superviseur.	Cahier des charges, spécifications, matrice de conformité
2	Modèle de ledger (comptes, écritures, journaux), machines à états, modèle de menaces STRIDE, OpenAPI, maquettes.	Architecture, ADR, modèle de menaces
3	Gradle multi-modules, Testcontainers, SonarQube, CI, environnement sandbox.	Guide développeur
4	Sprint 1 : identité et KYC · Sprint 2 : ledger et transferts · Sprint 3 : app mobile (onboarding, transferts) · Sprint 4 : QR marchand et notifications · Sprint 5 : anti-fraude et back-office.	Journal de sprints
5	Tests de concurrence (double dépense), tests d'intrusion, revue de code sécurité, tests de charge.	Stratégie de tests, rapport de pentest
6	Kubernetes ou VM durcies, secrets Vault, sauvegardes PITR, plan de reprise, procédure d'incident.	Guide de déploiement, runbook, PRA
7	Guides client, agent, marchand et superviseur ; politique de sécurité publiée.	Documentation générale, guide utilisateur

09 · Télémédecine : rendez-vous, visio et dossier patient

Type	Mobile multiplateforme + web praticien
Domaine	Santé
Niveau	Avancé
Durée indicative	10 semaines

Objectif et exigences

Mettre en relation patients et médecins : prise de rendez-vous, consultation vidéo, ordonnance électronique, dossier médical partagé. Exigences : confidentialité des données de santé (chiffrement au repos et en transit, journalisation des accès), vidéo fonctionnelle sur réseau mobile faible, disponibilité des créneaux en temps réel, accessibilité.

Stack et justification

Choix techniques du projet 09

Couche	Technologie	Pourquoi
Mobile	Dart · Flutter · Riverpod · flutter_webrtc	Une seule base pour Android et iOS, UI cohérente, bonne intégration WebRTC ; Flutter Web permet aussi une version navigateur pour le médecin.
Backend	Python · Django · Django REST Framework · Django Channels	Django apporte auth, admin et ORM ; Channels gère la signalisation WebRTC et les notifications WebSocket ; Python facilite les futurs modules d'aide au diagnostic.
Vidéo	WebRTC + serveur TURN (coturn) ou LiveKit	Vidéo pair-à-pair chiffrée ; TURN pour traverser les NAT ; LiveKit si consultations à plusieurs.
Données	PostgreSQL chiffré, S3 chiffré (documents), Redis	Dossiers, ordonnances, pièces jointes ; files et présence.
Interopérabilité	HL7 FHIR (ressources Patient, Appointment, Observation)	Standard international pour échanger avec d'autres systèmes de santé.

Fonctionnalités

- Annuaire de praticiens par spécialité, disponibilité, tarif ; agenda avec créneaux et rappels.
- Prise de rendez-vous, paiement de la consultation, salle d'attente virtuelle.
- Consultation vidéo / audio avec chat, partage de documents, adaptation à la bande passante.
- Dossier patient : antécédents, allergies, constantes, documents, historique des consultations.
- Ordonnance électronique signée en PDF, envoi à la pharmacie.
- Messagerie sécurisée asynchrone patient-médecin.
- Consentements, journal des accès au dossier, export des données (droit d'accès).
- Back-office : validation des diplômes, statistiques, facturation des praticiens.

Compétences acquises

- WebRTC (signalisation, ICE, TURN) et temps réel avec Django Channels.
- Flutter avancé : gestion d'état, navigation, permissions, notifications.
- Sécurité et conformité des données de santé, chiffrement, audit.
- Interopérabilité FHIR et conception d'API métier.
- Tests Flutter (widget, integration), pytest, tests de qualité vidéo.

Phases de développement

Phases spécifiques du projet 09

Phase	Étapes clés	Document
0-1	Cadre légal (secret médical, hébergement des données de santé), personas patient / médecin / secrétaire, parcours de consultation.	Cahier des charges, spécifications, analyse d'impact (AIPD)
2	Modèle FHIR simplifié, flux de signalisation, ADR WebRTC (P2P vs SFU), maquettes accessibles.	Architecture, ADR, OpenAPI, diagrammes de séquence
3	Monorepo Flutter + Django, Docker Compose avec coturn, CI (analyse, tests), flavors dev/staging.	Guide développeur
4	Sprint 1 : comptes et annuaire · Sprint 2 : agenda et rendez-vous · Sprint 3 : visio · Sprint 4 : dossier et ordonnances · Sprint 5 : messagerie, consentements, back-office.	Journal de sprints
5	Tests vidéo sur réseau dégradé (throttling), tests de permissions, pentest, tests d'accessibilité.	Stratégie de tests, rapport de sécurité
6	Hébergement certifié santé si disponible, TURN dédié, sauvegardes chiffrées, publication stores avec catégorie médicale.	Guide de déploiement, runbook
7	Guide patient (prendre rendez-vous, se connecter à la visio), guide praticien, charte de confidentialité.	Documentation générale, guide utilisateur

10 · Livraison de repas : client, livreur, restaurant

Type	3 applications mobiles + back-office web
Domaine	Logistique / restauration
Niveau	Avancé
Durée indicative	10 à 12 semaines

Objectif et exigences

Écosystème complet de commande et livraison : le client commande, le restaurant prépare, le livreur est attribué automatiquement et suivi en temps réel. Exigences : attribution du livreur le plus pertinent en moins de 5 secondes, suivi GPS toutes les 3 secondes, estimation du temps de livraison, calcul des frais par distance, gestion des annulations et des litiges.

Stack et justification

Choix techniques du projet 10

Couche	Technologie	Pourquoi
Mobile	Dart · Flutter (3 apps partageant un package commun) · Google Maps / Mapbox SDK · BLoC	Trois applications avec la même identité visuelle et beaucoup de logique commune (auth, cartes, notifications) : Flutter et un package partagé évitent de tripler le code.
Backend	Go · Gin ou Fiber · WebSocket (gorilla)	Latence faible pour les mises à jour de position et le dispatch ; excellente concurrence ; déploiement simple.
Géodonnées	PostgreSQL + PostGIS · Redis GEO	Redis GEO pour trouver instantanément les livreurs proches ; PostGIS pour les zones de livraison et l'historique.
Itinéraires	OSRM auto-hébergé ou API Mapbox Directions	Calcul de distance / durée et estimation de livraison.
Événements	NATS ou Kafka, FCM	Flux d'événements de commande et notifications push.
Web	React (back-office admin)	Supervision de la flotte et des restaurants.

Fonctionnalités

- App client : restaurants proches, menus avec options, panier, paiement, suivi en direct, notation.
- App restaurant : réception sonore des commandes, temps de préparation, gestion du menu et des ruptures.
- App livreur : mise en ligne, acceptation de courses, navigation, preuve de livraison (photo / code), revenus.
- Moteur de dispatch (distance, note, charge) avec réattribution automatique en cas de refus.
- Tarification dynamique par distance et heure, codes promo, pourboires.
- Chat client-livreur, appels masqués.
- Back-office : carte de la flotte en direct, litiges, remboursements, statistiques.

Compétences acquises

- Architecture multi-applications avec code partagé Flutter.
- Temps réel à grande échelle (WebSocket, Redis GEO, événements).
- Algorithmes de dispatch et de tarification, calcul d'itinéraires.
- Géolocalisation en arrière-plan et optimisation batterie.
- Go idiomatique, tests de charge, observabilité distribuée.

Phases de développement

Phases spécifiques du projet 10

Phase	Étapes clés	Document
0-1	Personas des trois acteurs, cycle de vie d'une commande, règles de tarification et d'annulation.	Cahier des charges, spécifications, diagramme d'états
2	Architecture événementielle, algorithme de dispatch (ADR), schéma PostGIS, contrats API et WebSocket, maquettes des 3 apps.	Architecture, ADR, OpenAPI / AsyncAPI
3	Monorepo Melos (Flutter) + Go modules, Docker Compose (Postgres, Redis, NATS, OSRM), CI.	Guide développeur
4	Sprint 1 : catalogue et commande · Sprint 2 : app restaurant · Sprint 3 : dispatch et app livreur · Sprint 4 : suivi temps réel · Sprint 5 : paiement, chat, back-office.	Journal de sprints
5	Simulateur de livreurs pour tests de charge, tests de dispatch, tests d'intégration bout en bout.	Stratégie de tests
6	Kubernetes ou Nomad, autoscaling, Grafana, publication des 3 apps.	Guide de déploiement, runbook
7	Trois guides utilisateurs distincts + guide administrateur.	Documentation générale, guides utilisateur

11 · Kanban collaboratif temps réel et hors ligne

Type	Application web (PWA)
Domaine	Productivité / gestion de projet
Niveau	Avancé
Durée indicative	8 semaines

Objectif et exigences

Outil type Trello / Jira léger : tableaux, colonnes, cartes, commentaires, pièces jointes, avec collaboration simultanée (curseurs, modifications visibles instantanément) et fonctionnement hors ligne. Exigences : latence perçue nulle (optimistic UI), fusion sans conflit des modifications concurrentes, historique et annulation, permissions par espace de travail, PWA installable.

Stack et justification

Choix techniques du projet 11

Couche	Technologie	Pourquoi
Backend	C# · ASP.NET Core 8 · SignalR · Entity Framework Core	SignalR fournit une abstraction temps réel robuste (WebSocket avec repli), groupes et reconnexion ; .NET est performant et très présent en entreprise.
Frontend	TypeScript · Angular 17 · NgRx · Angular CDK (drag and drop) · Yjs	Angular structure fortement les grandes applications ; NgRx pour un état prévisible ; Yjs (CRDT) pour fusionner les modifications concurrentes sans conflit, même hors ligne.
Données	SQL Server ou PostgreSQL · Redis (backplane SignalR)	Persistance relationnelle et diffusion entre instances.
Hors ligne	Service Worker (Angular PWA) · IndexedDB	Cache des tableaux et file d'opérations en attente.
Auth	ASP.NET Identity + OpenID Connect (Google, Microsoft)	Connexion d'entreprise.

Fonctionnalités

- Espaces de travail, tableaux, colonnes, cartes avec glisser-déposer fluide.
- Cartes riches : description Markdown, checklists, étiquettes, échéances, assignés, pièces jointes.
- Présence (avatars, curseurs), édition simultanée, notifications de mentions.
- Mode hors ligne avec synchronisation et résolution automatique (CRDT).
- Historique d'activité, annuler / rétablir, recherche globale.
- Automatisations simples (règles « quand… alors… »), rapports de flux (cumulative flow).
- Permissions (propriétaire, membre, invité), invitations, export JSON / CSV.

Compétences acquises

- Collaboration temps réel : SignalR, CRDT, optimistic UI.
- Angular avancé, gestion d'état NgRx, PWA et Service Workers.
- ASP.NET Core : middleware, EF Core, authentification OIDC.
- Conception de systèmes distribués côté client (offline-first).
- Tests xUnit, Jasmine / Karma, Cypress.

Phases de développement

Phases spécifiques du projet 11

Phase	Étapes clés	Document
0-1	Analyse de Trello / Linear, personas chef de projet / développeur, user stories de collaboration.	Cahier des charges, spécifications
2	Modèle de données, protocole d'événements SignalR, ADR CRDT (Yjs) vs OT, maquettes.	Architecture, ADR, contrat d'événements
3	Solution .NET + workspace Angular, Docker Compose, analyzers, ESLint, CI.	Guide développeur
4	Sprint 1 : tableaux et cartes · Sprint 2 : temps réel et présence · Sprint 3 : hors ligne et CRDT · Sprint 4 : permissions, historique, automatisations.	Journal de sprints
5	Tests de convergence CRDT (scénarios de conflits), tests de reconnexion, E2E Cypress.	Stratégie de tests
6	Azure App Service ou conteneurs, Redis backplane, Application Insights.	Guide de déploiement, runbook
7	Guide utilisateur avec GIF animés des interactions, FAQ hors ligne.	Documentation générale, guide utilisateur

12 · Portail e-gouvernement : démarches, files d'attente, paiements

Type	Application web institutionnelle
Domaine	Services publics
Niveau	Avancé
Durée indicative	10 semaines

Objectif et exigences

Guichet unique pour des démarches administratives (acte de naissance, permis, licences) : dépôt de dossier en ligne, suivi, rendez-vous, paiement des frais, délivrance de documents signés. Exigences : identité numérique unique, workflows de validation multi-services configurables, accessibilité (RGAA / WCAG AA), traçabilité, haute disponibilité, interopérabilité avec des systèmes existants.

Stack et justification

Choix techniques du projet 12

Couche	Technologie	Pourquoi
Backend	Java 21 · Spring Boot · Spring Batch · Flowable (moteur BPMN)	Stabilité et support long terme recherchés par les administrations ; un moteur BPMN permet de modéliser les circuits de validation sans coder chaque démarche.
Identité	Keycloak (OIDC, SAML)	SSO, fédération avec des annuaires existants, MFA, gestion des rôles centralisée.
Frontend	TypeScript · Angular · design system accessible	Formulaires complexes et dynamiques (Reactive Forms), i18n intégrée, structure adaptée aux grandes équipes.
Données	PostgreSQL · MinIO (pièces justificatives) · Elasticsearch (recherche de dossiers)	Fiabilité et recherche.
Documents	Signature électronique (PAdES) · QR de vérification	Actes délivrés vérifiables en ligne.

Fonctionnalités

- Compte citoyen unique, vérification d'identité, profil et documents réutilisables.
- Catalogue de démarches avec formulaires dynamiques générés depuis un schéma JSON.
- Dépôt de dossier, pièces justificatives, accusé de réception, suivi par étapes.
- Workflow BPMN d'instruction par les agents (assignation, demande de complément, validation, rejet motivé).
- Prise de rendez-vous et gestion de file d'attente en guichet (ticket, écran d'appel).
- Paiement des frais en ligne et quittance.
- Génération d'actes PDF signés avec QR code de vérification publique.
- Notifications e-mail / SMS, tableaux de bord de performance par service, données ouvertes.

Compétences acquises

- Modélisation de processus métier (BPMN) et moteur de workflow.
- Gestion d'identité d'entreprise (Keycloak, OIDC, SAML, MFA).
- Formulaires dynamiques et accessibilité web au niveau AA.
- Signature électronique et vérification de documents.
- Architecture Spring robuste, traitements batch, tests d'intégration.

Phases de développement

Phases spécifiques du projet 12

Phase	Étapes clés	Document
0-1	Cartographie de 3 démarches réelles, acteurs (citoyen, agent, chef de service), exigences d'accessibilité et de traçabilité.	Cahier des charges, spécifications, diagrammes BPMN
2	Architecture (Keycloak, Flowable, Angular), schéma des formulaires JSON, modèle de données, maquettes accessibles.	Architecture, ADR, OpenAPI
3	Maven multi-modules, Keycloak en Docker avec realm exporté, CI, tests d'accessibilité automatisés (axe).	Guide développeur
4	Sprint 1 : identité et catalogue · Sprint 2 : formulaires et dépôt · Sprint 3 : workflow agent · Sprint 4 : rendez-vous et paiement · Sprint 5 : actes signés, tableaux de bord.	Journal de sprints
5	Audit RGAA manuel, tests de workflow, tests de charge, pentest.	Stratégie de tests, rapport d'accessibilité
6	Haute disponibilité (2 nœuds + PostgreSQL répliqué), supervision, plan de continuité.	Guide de déploiement, runbook, PCA
7	Guide citoyen (avec version simplifiée), guide agent, guide administrateur de démarches.	Documentation générale, guides utilisateur

13 · Tableau de bord BI avec pipelines de données

Type	Web + ingénierie des données
Domaine	Analyse de données / décisionnel
Niveau	Avancé
Durée indicative	8 semaines

Objectif et exigences

Plateforme qui ingère des données hétérogènes (ventes CSV, API météo, base d'une application, logs), les nettoie, les agrège et les expose dans des tableaux de bord interactifs avec alertes. Exigences : pipelines planifiés et rejouables, qualité des données contrôlée, requêtes analytiques en moins d'une seconde sur 100 millions de lignes, constructeur de tableaux de bord sans code, partage et export.

Stack et justification

Choix techniques du projet 13

Couche	Technologie	Pourquoi
Orchestration	Python · Apache Airflow (ou Dagster) · pandas / Polars · dbt	Python est le langage de la donnée ; Airflow planifie et surveille les DAG ; dbt versionne les transformations SQL et leurs tests.
Entrepôt	ClickHouse (ou DuckDB pour démarrer)	Base orientée colonnes : agrégations ultra-rapides sur de gros volumes, compression, SQL standard.
API	FastAPI · SQLAlchemy · Redis (cache de requêtes)	Expose les requêtes analytiques paramétrées et le catalogue de métriques.
Frontend	TypeScript · React · Recharts / Apache ECharts · react-grid-layout	Constructeur de tableaux de bord par glisser-déposer et graphiques performants.
Qualité	Great Expectations · alertes Slack / e-mail	Validation des données et alerting.

Fonctionnalités

- Connecteurs : fichiers (CSV, Excel), bases SQL, API REST, dépôt S3 ; ingestion incrémentale.
- Pipelines Airflow avec reprise sur erreur, backfill, lignage des données.
- Modèle en couches (brut, nettoyé, métriques) avec tests dbt.
- Catalogue de métriques (définition unique, propriétaire, fraîcheur).
- Constructeur de tableaux de bord : graphiques, filtres croisés, drill-down, tableaux croisés.
- Alertes sur seuils et anomalies (détection statistique simple).
- Partage par lien, planification d'envoi PDF, export CSV, permissions par ligne.
- Page d'administration : état des pipelines, coûts, journaux.

Compétences acquises

- Ingénierie des données : ELT, orchestration, idempotence, backfill.
- Modélisation analytique (étoile, métriques), SQL avancé, bases colonnes.
- Qualité et observabilité des données.
- Visualisation de données efficace et UX de tableaux de bord.
- Tests de pipelines, dbt tests, pytest, Docker.

Phases de développement

Phases spécifiques du projet 13

Phase	Étapes clés	Document
0-1	Choix de 3 sources réelles ou ouvertes, questions métier à répondre, personas analyste / manager.	Cahier des charges, spécifications, catalogue des questions
2	Architecture des couches, modèle en étoile, ADR ClickHouse vs PostgreSQL, maquettes de tableaux de bord.	Architecture, ADR, dictionnaire des métriques
3	Docker Compose (Airflow, ClickHouse, Redis), projet dbt, CI avec tests dbt et pytest.	Guide développeur
4	Sprint 1 : ingestion et couche brute · Sprint 2 : transformations et métriques · Sprint 3 : API et graphiques · Sprint 4 : constructeur, alertes, partage.	Journal de sprints
5	Tests de qualité des données, tests de performance des requêtes, validation croisée des chiffres avec la source.	Stratégie de tests, rapport de validation
6	Déploiement sur VM ou Kubernetes, planification, sauvegarde de l'entrepôt.	Guide de déploiement, runbook des pipelines
7	Guide analyste (créer un tableau de bord), guide manager (lire et partager), documentation des métriques.	Documentation générale, guide utilisateur

14 · Plateforme de streaming vidéo

Type	Application web
Domaine	Médias
Niveau	Avancé
Durée indicative	9 semaines

Objectif et exigences

Service type YouTube simplifié : upload de vidéos, transcodage en plusieurs qualités, lecture adaptative, chaînes, abonnements, commentaires, recommandations. Exigences : upload résumable de fichiers de plusieurs Go, lecture sans coupure avec adaptation à la bande passante, protection des contenus privés (URL signées), sous-titres, modération.

Stack et justification

Choix techniques du projet 14

Couche	Technologie	Pourquoi
Frontend	TypeScript · Next.js · hls.js · Video.js	Pages vidéo indexables, lecteur HLS adaptatif dans le navigateur.
API	Node.js · Fastify · Prisma · PostgreSQL	I/O non bloquantes adaptées aux uploads et au streaming ; Fastify est rapide et typé.
Traitement	BullMQ (Redis) · FFmpeg · workers Node dédiés	Files de transcodage avec priorités, reprise et progression ; FFmpeg produit les renditions HLS (240p à 1080p), vignettes et sous-titres.
Stockage / diffusion	S3 compatible · CDN · URL signées	Stockage objet des segments, diffusion en périphérie, contrôle d'accès.
Upload	tus (protocole d'upload résumable)	Reprise après coupure pour les gros fichiers.
Recherche / reco	Meilisearch · calcul de recommandations par co-visionnage	Recherche et suggestions.

Fonctionnalités

- Upload résumable avec progression, file de transcodage et notification de disponibilité.
- Lecture HLS adaptative, reprise de lecture, vitesse, sous-titres, chapitres, miniatures de survol.
- Chaînes, abonnements, playlists, historique, « à regarder plus tard ».
- Commentaires imbriqués, likes, signalements, modération.
- Visibilité (public, non listé, privé) avec URL signées et expirantes.
- Statistiques créateur (vues, durée moyenne, rétention).
- Recherche et recommandations, page d'accueil personnalisée.
- Diffusion en direct (option) via RTMP → HLS.

Compétences acquises

- Traitement média (FFmpeg, HLS, codecs) et pipelines de jobs.
- Stockage objet, CDN, URL signées, coûts de bande passante.
- Uploads volumineux et protocoles résumables.
- Node.js performant (streams, backpressure) et workers.
- Analytics d'usage et systèmes de recommandation simples.

Phases de développement

Phases spécifiques du projet 14

Phase	Étapes clés	Document
0-1	Personas créateur / spectateur / modérateur, objectifs de qualité de lecture, budget de stockage.	Cahier des charges, spécifications
2	Pipeline de transcodage, échelle de renditions, modèle de données, ADR HLS vs DASH, maquettes.	Architecture, ADR, OpenAPI
3	Monorepo, Docker Compose (Postgres, Redis, MinIO, tusd), CI, jeu de vidéos de test.	Guide développeur
4	Sprint 1 : upload et transcodage · Sprint 2 : lecteur et pages vidéo · Sprint 3 : chaînes et social · Sprint 4 : privé, statistiques, recherche · Sprint 5 : reco, direct.	Journal de sprints
5	Tests de lecture sur réseaux simulés, tests de workers (échecs, reprises), coût par vidéo mesuré.	Stratégie de tests
6	Déploiement avec workers autoscalés, CDN, politique de rétention.	Guide de déploiement, runbook
7	Guide créateur (publier, analyser), guide spectateur, règles de la communauté.	Documentation générale, guide utilisateur

15 · Réseau social et messagerie temps réel

Type	Mobile multiplateforme + backend massivement concurrent
Domaine	Social / communication
Niveau	Expert
Durée indicative	12 semaines

Objectif et exigences

Application sociale avec profils, fil d'actualité, stories éphémères, messagerie instantanée individuelle et de groupe (texte, images, vocaux), présence et accusés de lecture. Exigences : 100 000 connexions simultanées visées, livraison des messages garantie et ordonnée, chiffrement de bout en bout des messages privés, fil d'actualité personnalisé, modération et blocage.

Stack et justification

Choix techniques du projet 15

Couche	Technologie	Pourquoi
Backend	Elixir · Phoenix · Phoenix Channels · Presence · Ecto	La machine virtuelle BEAM gère des millions de processus légers isolés : idéal pour des connexions WebSocket persistantes, la présence distribuée et la tolérance aux pannes (supervision). Phoenix Channels et Presence résolvent nativement des problèmes que d'autres stacks doivent réinventer.
Mobile	TypeScript · React Native (Expo) · Zustand · React Query · WatermelonDB	Une base pour iOS et Android avec accès natif via modules ; WatermelonDB pour un stockage local rapide des conversations ; grande communauté.
Données	PostgreSQL · Redis · S3 (médias)	Persistance, caches de fils, stockage des médias.
Chiffrement	Protocole Signal (libsignal) ou X3DH + Double Ratchet simplifié	Confidentialité de bout en bout des messages privés.
Notifications	APNs / FCM via Pigeon	Push fiables.

Fonctionnalités

- Profils, abonnements, publications (texte, images, vidéo courte), stories 24 h, réactions, commentaires.
- Fil d'actualité classé (récence + affinité) avec pagination infinie et cache local.
- Messagerie 1:1 et de groupe : saisie en cours, présence, accusés de lecture, réponses, messages vocaux, suppression.
- Chiffrement de bout en bout des conversations privées, vérification de clés.
- Recherche d'utilisateurs et de hashtags, suggestions d'amis.
- Signalement, blocage, filtres de contenu, outils de modération.
- Notifications push groupées et paramétrables, mode hors ligne (lecture, envoi différé).

Compétences acquises

- Programmation fonctionnelle et concurrente (Elixir, OTP, supervision).
- Systèmes temps réel à grande échelle : Channels, PubSub distribué, Presence.
- Cryptographie de bout en bout et gestion des clés.
- React Native avancé : listes performantes, médias, stockage local, push.
- Conception de fils d'actualité (fan-out, cache), tests ExUnit et Detox.

Phases de développement

Phases spécifiques du projet 15

Phase	Étapes clés	Document
0-1	Positionnement, personas, règles de communauté, exigences de confidentialité et de charge.	Cahier des charges, spécifications, modèle de menaces
2	Topologie des Channels, protocole de messages (versionné), stratégie de fil (push vs pull), ADR E2EE, maquettes.	Architecture, ADR, spécification du protocole
3	Projet Phoenix + app Expo, Docker Compose, Credo, ExUnit, CI, EAS Build.	Guide développeur
4	Sprint 1 : comptes et profils · Sprint 2 : publications et fil · Sprint 3 : messagerie temps réel · Sprint 4 : E2EE et médias · Sprint 5 : groupes, stories · Sprint 6 : modération, push, hors ligne.	Journal de sprints
5	Test de charge WebSocket (Tsung / k6), tests de livraison ordonnée, audit cryptographique, tests Detox.	Rapport de charge, stratégie de tests
6	Cluster Elixir (libcluster) sur Fly.io ou Kubernetes, releases, observabilité (Telemetry, Grafana), publication stores.	Guide de déploiement, runbook
7	Guide utilisateur, page « sécurité et confidentialité », FAQ modération.	Documentation générale, guide utilisateur

16 · Domotique IoT temps réel

Type	IoT (capteurs) + backend + web + mobile
Domaine	Maison connectée / énergie
Niveau	Expert
Durée indicative	10 à 12 semaines

Objectif et exigences

Plateforme qui collecte les données de capteurs (température, énergie, présence), pilote des actionneurs (relais, éclairage) et exécute des scénarios d'automatisation, avec tableau de bord web et application mobile. Exigences : ingestion de 10 000 messages/s, historique à haute résolution, règles évaluées en moins de 100 ms, fonctionnement local sans cloud (passerelle), sécurité des appareils (certificats), mises à jour à distance.

Stack et justification

Choix techniques du projet 16

Couche	Technologie	Pourquoi
Backend / passerelle	Rust · Actix Web · Tokio · rumqttc	Performance et sécurité mémoire pour un service qui tourne 24/7 sur une passerelle modeste (Raspberry Pi) ; concurrence asynchrone efficace pour ingérer des milliers de messages MQTT.
Messagerie	MQTT (Mosquitto ou EMQX) avec TLS et certificats par appareil	Standard IoT léger, publish/subscribe, QoS, sessions persistantes.
Séries temporelles	TimescaleDB (extension PostgreSQL)	Compression, agrégats continus et requêtes temporelles tout en gardant SQL et PostgreSQL.
Firmware	C++ (ESP32, PlatformIO) ou Rust embarqué	Capteurs réels ou simulateur logiciel ; OTA.
Web	TypeScript · SvelteKit · WebSocket · Chart.js	Tableau de bord léger et réactif, idéal pour l'affichage de flux en direct.
Mobile	Kotlin · Jetpack Compose (Android) · widgets	Contrôle rapide, notifications, géorepérage pour les scénarios « je rentre / je pars ».

Fonctionnalités

- Appairage sécurisé des appareils, provisionnement de certificats, inventaire et état de santé.
- Ingestion MQTT, validation des payloads, stockage haute résolution et agrégats (minute, heure, jour).
- Tableau de bord temps réel par pièce, historiques, consommation d'énergie et coût.
- Moteur de règles (conditions, horaires, géorepérage) avec éditeur visuel et journal d'exécution.
- Pilotage manuel des actionneurs, scènes, planification.

- Alertes (fuite, température, dépassement) par push et e-mail.
- Mode local hors internet ; synchronisation optionnelle vers le cloud.
- Mises à jour firmware OTA et gestion des versions.

Compétences acquises

- Rust asynchrone (Tokio), gestion des erreurs, performance.
- Protocoles IoT (MQTT, TLS mutuel), sécurité des appareils, OTA.
- Bases de séries temporelles, agrégats continus, rétention.
- Moteurs de règles et systèmes réactifs.
- Programmation embarquée de base, SvelteKit, Android.

Phases de développement

Phases spécifiques du projet 16

Phase	Étapes clés	Document
0-1	Cas d'usage (énergie, sécurité, confort), inventaire des capteurs, contraintes locales / hors ligne.	Cahier des charges, spécifications
2	Topologie MQTT (topics, QoS), schéma des payloads (JSON Schema), modèle TimescaleDB, DSL des règles (ADR), maquettes.	Architecture, ADR, spécification des topics
3	Cargo workspace, Docker Compose (Mosquitto, TimescaleDB), simulateur d'appareils, CI (clippy, tests), PlatformIO.	Guide développeur
4	Sprint 1 : ingestion et stockage · Sprint 2 : tableau de bord · Sprint 3 : actionneurs et scènes · Sprint 4 : moteur de règles · Sprint 5 : sécurité des appareils, OTA · Sprint 6 : mobile.	Journal de sprints
5	Test d'ingestion 10 000 msg/s, tests de latence des règles, tests de coupure réseau, revue sécurité TLS.	Rapport de performance, stratégie de tests
6	Image de passerelle (Raspberry Pi) reproductible, sauvegardes, supervision Grafana.	Guide de déploiement, runbook
7	Guide d'installation matérielle, guide utilisateur (règles, scènes), guide de dépannage.	Documentation générale, guide utilisateur

17 · Covoiturage / VTC avec suivi GPS

Type	Mobile multiplateforme (passager, conducteur) + backend temps réel
Domaine	Mobilité
Niveau	Expert
Durée indicative	12 semaines

Objectif et exigences

Application de mise en relation passager-conducteur : estimation de prix, recherche du conducteur proche, suivi de la course, paiement, notation, plus un mode covoiturage planifié (trajets publiés à l'avance). Exigences : mise en relation en moins de 10 secondes, positions rafraîchies chaque seconde pendant la course, tarification par distance et durée réelles, fonctionnement en zone de couverture faible, sécurité (partage de trajet, bouton d'urgence).

Stack et justification

Choix techniques du projet 17

Couche	Technologie	Pourquoi
Mobile	Kotlin Multiplatform (logique partagée) · Jetpack Compose (Android) · SwiftUI (iOS) · SDK cartes natifs	Partage du domaine, du réseau et du cache entre plateformes tout en gardant des interfaces 100 % natives et un accès complet au GPS en arrière-plan, critique pour ce type d'application.
Backend	Node.js · TypeScript · NestJS · Socket.IO	Excellente gestion de nombreuses connexions WebSocket, écosystème riche pour cartes et paiements ; NestJS structure les modules (courses, tarification, paiement).
Géodonnées	Redis GEO (positions en direct) · PostgreSQL + PostGIS (historique, zones) · H3 (indexation hexagonale pour la demande)	Recherche de proximité instantanée, analyses spatiales, tarification dynamique par zone.
Itinéraires	OSRM / Valhalla ou Mapbox Directions · map matching	Estimation, navigation et recalcul du kilométrage réel.
Paiement / autres	Stripe, mobile money · FCM / APNs · Twilio (appels masqués)	Encaissement, notifications, communication sécurisée.

Fonctionnalités

- Inscription conducteur avec vérification des documents, véhicule, disponibilité.
- Estimation du prix et du temps d'arrivée, options (économique, confort, partagé).
- Mise en relation par proximité et score, acceptation avec délai, réattribution.
- Suivi temps réel de l'approche et de la course, navigation conducteur, partage de trajet à un proche.
- Tarification dynamique (demande par zone H3), calcul final sur distance réelle, reçus.

- Covoiturage planifié : publication de trajets, réservation de places, correspondance d'itinéraires.
- Notation mutuelle, historique, portefeuille conducteur et reversements.
- Sécurité : bouton d'urgence, enregistrement du trajet, détection d'arrêt anormal.
- Back-office : supervision de la flotte, zones, tarifs, litiges.

Compétences acquises

- Kotlin Multiplatform et interopérabilité Swift, géolocalisation en arrière-plan sur deux OS.
- Systèmes géospatiaux temps réel (Redis GEO, H3, map matching).
- Algorithmes de mise en relation et de tarification dynamique.
- WebSocket à grande échelle, reconnexion, files de messages.
- Tests avec simulateur de flotte, tests sur appareils réels, publication double store.

Phases de développement

Phases spécifiques du projet 17

Phase	Étapes clés	Document
0-1	Personas passager / conducteur / opérateur, cycle de vie d'une course, règles de tarification et d'annulation, exigences de sécurité.	Cahier des charges, spécifications, diagramme d'états
2	Architecture modules NestJS, protocole Socket.IO versionné, ADR KMP, ADR Redis GEO + H3, maquettes.	Architecture, ADR, AsyncAPI / OpenAPI
3	Projet KMP + apps natives, backend NestJS, Docker Compose (Postgres/PostGIS, Redis, OSRM), CI multi-OS.	Guide développeur
4	Sprint 1 : comptes et cartes · Sprint 2 : estimation et demande · Sprint 3 : mise en relation et suivi · Sprint 4 : course, paiement, notation · Sprint 5 : covoiturage planifié · Sprint 6 : sécurité, back-office.	Journal de sprints
5	Simulateur de 1 000 conducteurs, tests de latence, tests GPS en conditions réelles, audit sécurité.	Stratégie de tests, rapport de charge
6	Kubernetes avec Socket.IO adapter Redis, autoscaling, publication App Store / Play Store.	Guide de déploiement, runbook
7	Guides passager et conducteur, charte de sécurité, guide opérateur.	Documentation générale, guides utilisateur

18 · Réservation de voyages en microservices

Type	Application web distribuée
Domaine	Voyage / hôtellerie / transport
Niveau	Expert
Durée indicative	12 semaines

Objectif et exigences

Plateforme de réservation combinant hôtels, vols (simulés via fournisseurs fictifs) et activités, avec panier multi-produits et paiement unique. Exigences : réservation atomique de plusieurs services chez des fournisseurs distincts (saga avec compensation), disponibilité 99,9 %, recherche agrégée en moins de 2 secondes, résilience aux pannes d'un fournisseur, observabilité complète.

Stack et justification

Choix techniques du projet 18

Couche	Technologie	Pourquoi
Services	Java 21 · Spring Boot · Spring Cloud (Gateway, Config, Resilience4j) · un service par domaine (recherche, hôtel, vol, réservation, paiement, notification)	Écosystème microservices le plus mature : découverte, configuration centralisée, circuit breakers, retries. Chaque service a sa propre base de données.
Messagerie	Apache Kafka · pattern Saga (orchestration) · Outbox	Coordination des réservations multi-fournisseurs avec compensation en cas d'échec ; livraison fiable des événements.
Données	PostgreSQL par service · Redis (cache de disponibilité) · Elasticsearch (recherche)	Isolation, performance de recherche.
Frontend	TypeScript · React · Next.js	Parcours de recherche et de réservation, SEO des pages destination.
Infrastructure	Docker · Kubernetes (k3s en local) · Helm · GitHub Actions · Terraform	Déploiement déclaratif, montée en charge par service.
Observabilité	OpenTelemetry · Jaeger · Prometheus · Grafana · Loki	Traces distribuées indispensables pour déboguer une saga.

Fonctionnalités

- Recherche unifiée (destination, dates, voyageurs) agrégeant plusieurs services en parallèle avec délais d'attente.
- Fiches hôtel / vol / activité, disponibilité et prix en temps réel, cache intelligent.
- Panier multi-produits, réservation atomique par saga avec compensation automatique.
- Paiement unique, gestion des échecs partiels et remboursements.
- Espace client : réservations, modifications, annulations selon conditions, documents (vouchers PDF).

- Notifications e-mail / SMS, rappels avant le départ.
- Espace fournisseur : gestion de l'inventaire, tarifs, règles d'annulation.
- Tableau de bord d'exploitation : santé des services, sagas en cours, alertes.

Compétences acquises

- Architecture microservices, découpage par domaine (DDD), API Gateway.
- Transactions distribuées (Saga, Outbox), Kafka, idempotence.
- Résilience (circuit breaker, retry, bulkhead, timeouts).
- Kubernetes, Helm, Terraform, CI/CD par service.
- Observabilité distribuée, tests de contrat (Pact), tests de chaos.

Phases de développement

Phases spécifiques du projet 18

Phase	Étapes clés	Document
0-1	Personas voyageur / fournisseur / opérateur, parcours de réservation, exigences de disponibilité, fournisseurs fictifs à simuler.	Cahier des charges, spécifications
2	Event storming, carte des contextes (DDD), diagramme de la saga, contrats d'événements (AsyncAPI) et d'API, ADR orchestration vs chorégraphie.	Architecture, ADR, catalogue d'événements
3	Dépôt par service ou monorepo, Docker Compose puis k3s, Helm charts, CI par service, environnement local complet.	Guide développeur
4	Sprint 1 : gateway et recherche · Sprint 2 : services hôtel et vol · Sprint 3 : saga de réservation · Sprint 4 : paiement et compensation · Sprint 5 : espace client et fournisseur · Sprint 6 : observabilité et résilience.	Journal de sprints
5	Tests de contrat, tests de chaos (arrêt d'un service), tests de charge, vérification de la cohérence des sagas.	Stratégie de tests, rapport de résilience
6	Cluster Kubernetes, déploiements canari, sauvegardes par service, alertes SLO.	Guide de déploiement, runbook par service
7	Guide voyageur, guide fournisseur, documentation d'exploitation.	Documentation générale, guides utilisateur

19 · Jeu mobile multijoueur en temps réel

Type	Jeu mobile (Android, iOS) + serveur de jeu
Domaine	Jeu vidéo / divertissement
Niveau	Expert
Durée indicative	12 semaines

Objectif et exigences

Jeu de quiz et de réflexes en arène (4 à 8 joueurs) avec parties classées, saisons, boutique cosmétique et classements. Exigences : synchronisation des joueurs avec une latence inférieure à 150 ms, serveur autoritaire (anti-triche), matchmaking par niveau en moins de 15 secondes, reconnexion en cours de partie, monétisation équitable, 60 images par seconde sur appareils moyens.

Stack et justification

Choix techniques du projet 19

Couche	Technologie	Pourquoi
Client	C# · Unity 2022 LTS · UI Toolkit · Addressables	Moteur de jeu de référence pour le mobile : rendu, animations, gestion des assets, export Android et iOS depuis un seul projet ; C# solide et typé.
Serveur de jeu	Go · WebSocket (nhooyr) · boucle de jeu à tick fixe · Protobuf	Goroutines pour des milliers de salles simultanées, faible latence, binaire léger à répliquer ; Protobuf pour des messages compacts.
Services	Go ou Node.js : matchmaking, profils, classements, boutique · Redis (sorted sets) · PostgreSQL	Classements instantanés, files de matchmaking, persistance des comptes et achats.
Monétisation	Unity IAP, validation serveur des reçus · publicités récompensées	Achats sécurisés et vérifiés côté serveur.
Backend as a service (option)	Nakama (open source) ou PlayFab	Comparer un serveur maison à une solution existante et documenter le choix.

Fonctionnalités

- Comptes (invité, Google, Apple), profil, avatar, progression et niveaux.
- Matchmaking par classement (Elo / Glicko), salles privées entre amis.
- Parties temps réel : questions synchronisées, mini-jeux de réflexes, bonus, chat rapide par émoticônes.
- Serveur autoritaire : validation des réponses, chronométrage, détection de triche, reconnexion.
- Classements quotidiens / hebdomadaires / saisonniers, récompenses de saison.
- Boutique cosmétique, monnaie virtuelle, quêtes journalières, passe de saison.
- Éditeur de contenu (questions, catégories) et A/B tests de difficulté.
- Télémétrie de jeu (rétention, durée de session, taux d'abandon).

Compétences acquises

- Développement de jeu Unity : boucle de jeu, UI, animations, optimisation mobile.
- Réseau temps réel de jeu : serveur autoritaire, prédiction, réconciliation, ticks.
- Matchmaking et systèmes de classement.
- Économie de jeu, achats intégrés sécurisés, télémétrie.
- Go concurrent, Protobuf, tests de charge de salles, publication stores.

Phases de développement

Phases spécifiques du projet 19

Phase	Étapes clés	Document
0-1	Game design document (boucle de jeu, règles, économie), personas joueurs, objectifs de performance.	Cahier des charges, game design document
2	Architecture client-serveur, protocole Protobuf, machine à états d'une partie, ADR serveur maison vs Nakama, maquettes et prototype papier.	Architecture, ADR, spécification du protocole
3	Projet Unity avec assembly definitions, serveur Go, Docker, CI (Unity Cloud Build ou GameCI), tests de protocole.	Guide développeur
4	Sprint 1 : prototype jouable local · Sprint 2 : serveur autoritaire et salle · Sprint 3 : matchmaking et comptes · Sprint 4 : classements et saisons · Sprint 5 : boutique et IAP · Sprint 6 : polish, tutoriel, télémétrie.	Journal de sprints
5	Bots de charge (10 000 joueurs simulés), tests de latence artificielle, profilage FPS, playtests avec 20 joueurs.	Rapport de charge, rapport de playtest
6	Serveurs de jeu multi-régions (Agones ou autoscaling simple), CDN d'assets, publication stores, soft launch.	Guide de déploiement, runbook
7	Tutoriel intégré, guide joueur, notes de saison, FAQ achats.	Documentation générale, guide utilisateur

20 · Assistant IA de service client (RAG)

Type	Web (widget intégrable + back-office) + API
Domaine	Intelligence artificielle / relation client
Niveau	Expert
Durée indicative	10 semaines

Objectif et exigences

Assistant conversationnel qui répond aux clients d'une entreprise à partir de sa documentation (PDF, pages web, FAQ, tickets), cite ses sources, exécute des actions (suivre une commande, créer un ticket) et transfère à un humain si nécessaire. Exigences : réponses ancrées dans les sources (pas d'invention), streaming des réponses, multilingue, isolation des données par client, coût par conversation maîtrisé, évaluation continue de la qualité.

Stack et justification

Choix techniques du projet 20

Couche	Technologie	Pourquoi
Backend IA	Python 3.12 · FastAPI · LangChain ou LlamaIndex · Pydantic	Python concentre l'écosystème IA ; FastAPI gère nativement le streaming (SSE / WebSocket) et l'asynchrone ; les frameworks RAG fournissent découpage, embeddings, récupération et outils.
Modèles	API LLM (OpenAI, Mistral, Anthropic) avec abstraction · modèle open source local (Ollama) en option	Comparer coût / qualité et documenter le choix ; éviter le verrouillage fournisseur.
Vecteurs	PostgreSQL + pgvector · recherche hybride (BM25 + vecteurs) · reranking	Une seule base pour données et vecteurs, isolation par tenant, meilleure pertinence avec la recherche hybride.
Ingestion	Celery ou Arq · Unstructured / pypdf · crawler (Playwright)	Pipeline d'extraction, découpage, embeddings, mise à jour incrémentale.
Frontend	TypeScript · React · widget compilé en Web Component (Vite) · back-office Next.js	Widget intégrable sur n'importe quel site en une ligne de script ; back-office pour les sources, conversations et évaluations.
Évaluation	Ragas / jeux de tests maison · Langfuse (traces, coûts)	Mesurer fidélité, pertinence, hallucinations et coût par réponse.

Fonctionnalités

- Connexion de sources : upload de documents, crawl de site, import de FAQ et de tickets ; resynchronisation planifiée.
- Pipeline RAG : découpage sémantique, embeddings, recherche hybride, reranking, prompt avec citations.

- Chat en streaming avec citations cliquables, suggestions de questions, historique de conversation.
- Outils / actions : suivi de commande via API cliente, création de ticket, prise de rendez-vous (function calling).
- Transfert à un agent humain avec résumé de la conversation ; boîte de réception agent.
- Garde-fous : refus hors périmètre, détection de données sensibles, limites de coût.
- Back-office : sources, conversations, notes des utilisateurs, tableau de bord de qualité et de coûts.
- Multi-tenant, clés d'API, widget personnalisable (couleurs, langue, persona).
- Évaluation automatique sur un jeu de questions de référence à chaque changement de prompt ou de modèle.

Compétences acquises

- Ingénierie LLM : RAG, embeddings, recherche hybride, prompt engineering, function calling.
- Évaluation et observabilité des systèmes IA (métriques, traces, coûts).
- Streaming temps réel (SSE) et conception de widgets embarquables.
- Pipelines d'ingestion de documents et bases vectorielles.
- Sécurité des applications IA (injection de prompt, fuites de données), multi-tenant.

Phases de développement

Phases spécifiques du projet 20

Phase	Étapes clés	Document
0-1	Cas d'usage d'une entreprise fictive, corpus de documents, 100 questions de référence avec réponses attendues, personas client / agent / administrateur.	Cahier des charges, spécifications, jeu d'évaluation
2	Architecture du pipeline RAG, schéma pgvector, ADR choix du modèle et de la stratégie de découpage, conception des prompts, maquettes du widget.	Architecture, ADR, catalogue de prompts
3	Docker Compose (Postgres/pgvector, Redis, Langfuse), uv, ruff, pytest, CI avec évaluation automatique.	Guide développeur
4	Sprint 1 : ingestion et recherche · Sprint 2 : génération avec citations et streaming · Sprint 3 : widget et back-office · Sprint 4 : actions et transfert humain · Sprint 5 : garde-fous, multi-tenant, évaluation continue.	Journal de sprints
5	Évaluation Ragas (fidélité, pertinence), tests d'injection de prompt, tests de charge du streaming, mesure du coût par conversation.	Rapport d'évaluation, stratégie de tests
6	Déploiement conteneurisé, quotas et alertes de coût, rotation des clés, journalisation conforme.	Guide de déploiement, runbook
7	Guide d'intégration du widget, guide administrateur (sources, qualité), guide agent, FAQ sur les limites de l'IA.	Documentation générale, guides utilisateur

Synthèse des compétences à mettre en avant

Compétences transversales et projets qui les démontrent

Compétence	Projets
Temps réel (WebSocket, présence, CRDT)	10, 11, 15, 16, 17, 19
Paiements et conformité financière	03, 06, 07, 08, 12, 17, 18
Géolocalisation et données spatiales	02, 04, 10, 17
Hors ligne et synchronisation	04, 05, 06, 11, 15
Sécurité avancée (chiffrement, signatures, modèle de menaces)	06, 08, 09, 15, 16, 20
Données, IA et apprentissage automatique	05, 13, 14, 20
Architecture distribuée et DevOps (Kubernetes, Kafka, observabilité)	08, 10, 14, 18, 19
Mobile natif (Kotlin, Swift, KMP)	04, 05, 08, 16, 17
Mobile multiplateforme (Flutter, React Native)	06, 09, 10, 15
Accessibilité et conformité réglementaire	01, 03, 09, 12

Risque principal : la dispersion. Vingt projets complets représentent plus d'un an de travail à temps plein. Livrez chaque projet jusqu'à la phase 7 (documentation et guide utilisateur) avant d'en commencer un autre : un projet documenté et déployé vaut davantage sur un portfolio que trois prototypes inachevés.

\+ Modèle de fiche portfolio à produire pour chaque projet terminé

Durées indicatives pour un développeur seul à temps plein. Les versions de frameworks citées sont celles courantes en 2025-2026 ; adaptez-les au moment du démarrage. Document produit par Devin.